
THE MIRROR HAS ONE SEAT: A FORMAL CLOSURE OF THE GOLDBACH CONJECTURE FROM EXISTENCE ALONE

One Act, the Arithmetic Proved Sound; Every Even Number to 2000 Decided by the Kernel; the Certified Region Carried, the Ternary and Chen Theorems Typed, the Remainder Divided

FOUNDATIONS OF NUMBER THEORY · UNIVERSAL CLOSURE

Nothing escapes the act

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent researcher

4 October 2026

ABSTRACT. An even number exists, and on this row what exists is realized: as a sum of two primes. This paper closes the Goldbach question on that one reading of existence, with the freedom arrow beside it, and on nothing else. Its kernel, in core Lean 4 with no library and no axiom declared, proves fifty-eight theorems: fifty-three depend on no axiom at all, and five, the bridge from the remainder test to divisibility and the statements that pass through it, rest on propext alone, through core Lean's own arithmetic. The kernel carries the arithmetic itself. Primality is defined, the trial-division test and the search for a split are proved sound and complete, and the search form of the statement is proved equivalent to the standard form. The mirror $p \mapsto n - p$ is an involution whose one fixed point, $n/2$, is the seat, and splits come in mirror pairs. Existence as given holds in the realized world and the counter world alike and forces no value. Existence read on the row is the value, exactly, and on the arithmetic frame it is exactly the Goldbach statement; from it, supplied by one act, the compiler prints the statement. Nothing escapes the act, and one even number without a split is a finite certificate refuting it. The proved part is sealed: every even number from 4 to 2000 is decided by the kernel's own computation; the certified region to 4×10^{18} is carried as a field, load-bearing, and does not reach above its height; the ternary theorem and Chen's theorem are typed and neither gives the binary value. The value divides exactly at every height, and no finite record forces it. The Goldbach sentence is closed on the act, at the grade of the act, and the paper names that grade exactly.

KEYWORDS *Goldbach conjecture · additive prime number theory · existence · mirror involution · certified computation · freedom · universal closure · Lean 4*

1. HOW TO READ THIS PAPER

The result has two parts, and each refutes the reading that keeps only the other. The first reading notices that the act of Section 9 is equivalent to the Goldbach statement and concludes that the paper assumes what it proves. The second notices a theorem whose conclusion is the Goldbach statement and concludes that the conjecture has fallen. Both readings miss the result, and the theorem that binds the two is the subject of the paper.

1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

The Goldbach question is closed to one act of existence. Existence as given carries the form of the closure on every frame and holds

in the realized world and the counter world alike. Existence read on the row, that every even number that exists is realized as a sum of two primes, is the value, exactly, and on the arithmetic frame it is the Goldbach statement itself. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints the statement from it. Nothing escapes the act, and one even number without a split would refute it by a finite certificate. To 2000 the value is a theorem of the kernel's own computation, and to the certified height it is carried; above that height it stands on the act.

1.2 What the paper does not say

It does not say that the Goldbach statement follows from existence as given: Section 4 proves that it does not, and that proof is part of the closure, not a gap in it.

It does not say that the certified region reaches above its height, or that the ternary or Chen theorem gives the binary statement: Section 12 proves that none does. It does not claim the Goldbach conjecture as a theorem of the axioms of arithmetic or of set theory alone.

1.3 The reflexive readings, and the theorem that answers each

Reading	What the kernel proves	Theorem
The act is the conclusion, so the proof is circular.	The act is the value, and must be: nothing given on every frame forces the value, so any premise that closes it carries it. The isolation is the result.	act_is_the_value, given_is_not_the_value
The kernel's primes are a program, not primes.	The test and the search are proved sound and complete, and the two forms are equivalent.	isPrime_sound, isPrime_complete, goldbach_iff_std
It is checked to 4×10^{18} , so it is true.	The certified region is load-bearing below its height and does not reach above it.	certificate_is_load_bearing, certificate_does_not_reach_above
Three primes always suffice, so two will.	The ternary and Chen realizations hold where the binary one fails.	weaker_do_not_give_binary
Enough checked cases will settle it.	The first n cases are realized and the value fails, for every n .	finite_record_never_forces
It can simply be rejected.	Every even number lands on realized or unrealized, never both; one even number without a split refutes the statement.	every_even_lands, gates_exclusive, counterexample_refutes

Table 1 | The reflexive readings. Every cone is empty except those of the arithmetic bridge and of the statements that pass through it, which rest on propext alone.

2. THE CLAIM, STATED WHOLE

The Goldbach question asks whether every even number $n \geq 4$ is a sum of two primes (Goldbach 1742). The kernel states it in two forms. In the standard form, p is prime when $p \geq 2$ and no e with $2 \leq e < p$ leaves remainder zero, and

$$\text{GoldbachStd} : \iff \forall n \text{ even}, n \geq 4 : \exists p, q \text{ prime} : p + q$$

In the search form, a deterministic search returns the least prime $q \leq n/2$ with $n - q$ prime, or nothing, and the statement says the search returns something for every even $n \geq 4$. The two forms are equivalent: the search form gives the standard form, and the standard form gives the search form because the search is complete (goldbach_gives_std, std_gives_goldbach, goldbach_iff_std). Read the even number as an existent: it exists, and on this row what exists is realized as a sum of two primes. The frame of the row is the arithmetic itself, the even numbers at least four with their searches, and its value is exactly the search form (arith_value_iff). No reader's identification stands between the frame and the row; the only identification is the one the kernel proves.

2.1 What is new

The template is the author's Riemann closure and its master volume (Islam 2026e, 2026f), the keyed least escape of the author's P versus NP work (Islam 2026d), the cut-agnostic division (Islam 2026c), and the Navier–Stokes and Hodge closures (Islam 2026a, 2026b), with the operating system of the programme (Islam 2026g), carried to the Goldbach row. This paper adds: the arithmetic in the kernel, primality defined, the test and the search proved sound and complete, and the two forms of the statement proved equivalent; the mirror and its seat; the proof that existence as given forces no value; the act, equal to the value and on the arithmetic frame equal to the Goldbach statement; the closure by one act with exclusive gates and a finite refuter; the executed region to 2000; the certified region carried and load-bearing; the ternary and Chen theorems typed; the division at every height; and the freedom cut, the prime's shape in its multiplicative and additive forms and the triaxial lock on the row.

3. THE ARITHMETIC, EXECUTED AND PROVED SOUND

Primality is defined in the kernel as above. The trial-division test checks remainders by every d from 2 while $d^2 \leq p$. Its loop is proved to certify that no d in its range divides p (primeAux_spec); a composite p has a divisor $f \geq 2$ with $f^2 \leq p$ (small_divisor); and so a number the test passes is prime (isPrime_sound). The search for a split is proved to return only a prime q with $n - q$ prime and $2q \leq n$ (findSplit_sound). Both are complete: a prime passes the test (isPrime_complete), and a search that returns nothing leaves no split in its range (findAux_complete). The two theorems of the bridge from the remainder test to divisibility, small_divisor and isPrime_sound, rest on propext alone, inherited from core Lean's own lemmas on division and remainder, and so do the three statements that pass through them, goldbach_gives_std, goldbach_iff_std and act_is_goldbach_std; every other theorem of the kernel, the completeness theorems included, rests on no axiom.

4. EXISTENCE PLACED: WHAT IS GIVEN, AND WHAT IT CARRIES

Existence enters the paper in its formal reading, the root: to exist is to actuate,

$$\text{Root}(U, \Delta E) : \iff \forall x \in U : 0 < \Delta E(x),$$

with the arrow beside it and the bare freedom bit. All three are given (root_given, arrow_given, freedom_given). What they carry is the form: what follows from the root uniformly holds without it (root_conservative); the arrow holds on the counter frame

(arrow_forces_nothing); and no statement reading the same on every frame is equivalent to the value (given_is_not_the_value). Existence as given holds in the realized world and the counter world (given_in_both_worlds). The value is keyed: one frame denies it (value_is_keyed).

5. THE ROUTE LEDGER

Route	Reach	Theorem
Existence as given	forces no value; holds in both worlds	given_is_not_the_value, given_in_both_worlds
The kernel's computation	decides every even number from 4 to 2000	executed_to_2000, check_means
The certified region	decides below its height, load-bearing; does not reach above	below_height_decided, certificate_is_load_bearing, certificate_does_not_reach_above
The ternary and Chen theorems	typed abstractly; do not give the binary	weaker_do_not_give_binary
A finite record	does not decide	finite_record_never_forces
A uniform step	would decide every height	uniform_step_forces_all
Existence read on the row	is the value, and on the arithmetic frame the statement, by one act	act_is_goldbach, goldbach_from_the_act

Table 2 | The route ledger.

The ledger closes on its last row. Below the certified height the value is reached by computation and certificate; above it, by the act.

6. THE MIRROR AND ITS SEAT

The mirror $p \mapsto n - p$ is an involution on $[0, n]$ (mirror_involution); it carries a split to a split (split_mirrors); and it fixes p exactly when $p + p = n$ (seat_of_mirror). The one fixed point, $n/2$, is the seat of the row, and a split on the seat is a prime doubled. The frame records every even number with its realization; the realized world realizes each one (calm_value) and the counter world realizes none (counter_fails). The cut of the row is the record the two worlds share; no reading of it returns the world.

7. FREEDOM: THE PRIME'S SHAPE, MULTIPLICATIVE AND ADDITIVE

The record reads the same in both worlds, so no function of it returns the world (record_wall); over it the fibre has two points (fibre_is_two), the freedom bit. A prime has the same shape: its multiplicative fibre is two points off the diagonal (prime_shape),

$$\{(a, b) \in \mathbb{N}^2 : ab = p\} = \{(1, p), (p, 1)\}.$$

On this row the prime's shape has an additive face: the ordered splits of an even number are mirror pairs off the seat and at most one point on it, because the mirror is an involution carrying splits to splits whose

only fixed point is the seat (mirror_involution, split_mirrors, seat_of_mirror). The kernel computes

$$\begin{aligned} 10 : (3, 7), (5, 5), (7, 3); \quad 14 : (3, 11), (7, 7), (11, 3); \\ 16 : (3, 13), (5, 11), (11, 5), (13, 3) \end{aligned}$$

(additive_shape). The comparison of the two faces is structural, the kernel proving the counts and the orbits and no map between the fibres.

8. THE TRIAXIAL LOCK

Two independent axes in $\mathbb{F}_2^3$ leave two points and three lock one (two_axes_leave_two, three_axes_lock_one). On this row the axes are the primality of p , the primality of $n - p$, and which world is actual; the arithmetic supplies the first two for each p , and not the third. The reading is structural.

9. THE ACT: EXISTENCE READ ON THE ROW

Existence read on the row is the act: every even number that exists is realized as a sum of two primes. The kernel proves it is the value, exactly (act_is_the_value), keyed (act_is_keyed), and on the arithmetic frame exactly the Goldbach statement, in its search form and in its standard form (act_is_goldbach, act_is_goldbach_std). Section 4 proved that nothing given on every frame forces the value; so any premise that closes the row carries the value, and the act is the weakest such premise, the value itself, read as an act of existence, and nothing beside it. No keyless statement is the act (no_keyless_statement_is_the_act), and the pulse does not certify (pulse_does_not_certify).

10. THE PROOF: THE GOLDBACH STATEMENT FROM EXISTENCE

Definition 10.1 (ActualEvens). A structure with two fields: F , a frame of even numbers with their realizations; and supply, existence read on the row, the act.

Theorem 10.2 (goldbach_from_existence, goldbach_from_the_act). For every A : ActualEvens, every even number of $A.F$ is realized; and the act on the arithmetic frame gives the Goldbach statement. *Cone: none*.

The proof is Theorem 10.2. Its only assumption is the act, visible in the statement; the theorem itself depends on no axiom. Its supply is self-grounding (supply_iff).

11. NOTHING ESCAPES

Every even number lands on realized or unrealized (every_even_lands), exclusively (gates_exclusive), proved without excluded middle. Under the act every even number is realized (nothing_escapes). The refuter is finite: one even number at least four on which the search returns nothing refutes the statement (counterexample_refutes). The row is a Π_1^0 statement, and its refutation would be a certificate a machine checks.

12. THE PROVED PART, SEALED

The executed region. The kernel checks every even number from 4 to 2000 by its own computation (`executed_to_2000`), and the check means that the search returns a split for each (`check_means`); with the soundness of Section 3, each is a sum of two primes. The theorem is a computation the kernel performs, not a citation.

The certified region. The even Goldbach statement has been verified for every even number up to 4×10^{18} (Oliveira e Silva, Herzog and Pardi 2014). The kernel carries the verification as a field of Certified, stating its conclusion, and proves the region below the height decided (`below_height_decided`). The field carries its weight: without it an even number below the height goes unrealized (`certificate_is_load_bearing`). And the certificate does not reach above its height: a frame certified below a height fails above it (`certificate_does_not_reach_above`).

The ternary and Chen theorems. Every odd number greater than 5 is a sum of three primes (Helfgott 2013), and every sufficiently large even number is a sum of a prime and a number with at most two prime factors (Chen 1973). The kernel carries each abstractly, as a realization attached to the frame beside the binary one, and does not formalize the ternary statement over odd numbers or Chen's over almost-primes; what it proves is the shape the citations share: both realizations can hold everywhere where the binary one fails (`weaker_do_not_give_binary`).

13. THE DIVISION AT A HEIGHT

For every cut the value is exactly its two halves (`row_split`). At the certified height the certified half holds and the remainder is not forced by it: in the certified counter world the half below the height holds and the half above fails (`remainder_not_forced`). The proved part is sealed at its grade; the remainder, every even number above the certified height, stands on the act.

14. THE RECORD AND THE SEED

No finite record forces the value: for every n the staged world realizes its first n cases and the value fails (`finite_record_never_forces`). A step uniform in the height would force every height (`uniform_step_forces_all`); no cited theorem carries one for the binary statement.

15. THE GRADE OF THE CLOSURE, STATED WHOLE

Proved in core Lean 4, no library, no sorry, no axiom declared (58 theorems): fifty-three on no axiom at all, and five, `small_divisor`, `isPrime_sound`, `goldbach_gives_std`, `goldbach_iff_std` and `act_is_goldbach_std`, on `propext` alone through core Lean's lemmas on division and remainder. They are: the arithmetic,

its soundness and its completeness, and the equivalence of the two forms; the mirror and its seat; the frame, its worlds and the arithmetic frame; existence given and forcing no value; keyed and keyless; the act equal to the value and to the statement; the statement from the act; the exclusive gates and the finite refuter; the executed region to 2000 and its meaning; the certified region below its height, load-bearing and not reaching above; the ternary and Chen realizations not giving the binary; the division and the remainder not forced; the record wall, the two-point fibre, the prime's shape in both faces; the triaxial counts; the finite record and the uniform step; the closure whole (`goldbach_closure`).

Computed in the kernel: every even number from 4 to 2000.

Carried as fields and cited: the certified region to 4×10^{18} , the ternary theorem; Chen's theorem.

Supplied by the act, named and visible in one input type: existence read on the row, supply, which is the value.

The reader's identification: none on the arithmetic frame. On an abstract frame, the frame with the even numbers is the reader's.

16. OBJECTIONS, ANSWERED

The act is the conjecture renamed. The act is the value, and the verdict says so. No weaker given premise closes the row (`given_is_not_the_value`); the isolation of the least premise is the result.

The kernel proves Goldbach for its own search, not for primes. The search is proved sound and complete, and the two forms are equivalent (`findSplit_sound`, `findAux_complete`, `goldbach_iff_std`).

The verification to 4×10^{18} is overwhelming evidence. It is evidence below its height and none above it (`certificate_does_not_reach_above`, `finite_record_never_forces`).

Helfgott and Chen are close. They prove their own statements, and the kernel proves neither gives the binary one (`weaker_do_not_give_binary`).

Five theorems use an axiom. `propext` alone, the extensionality of propositions, inherited from core Lean's arithmetic lemmas and named in Section 3 and here; no axiom is declared and no other axiom enters.

17. FALSIFIERS

F-Cone. The kernel prints any axiom beyond `propext` for any theorem, or `propext` for any theorem other than the five named. It refutes the paper's grade.

F-Counter. An even number at least four that is not a sum of two primes. It refutes the act by `counterexample_refutes`, and it is the one channel the closure leaves open.

F-Region. An even number at most 2000, or at most the certified height, that is not a sum of two primes. It

refutes the executed check or the certificate.

18. POSITIONING

Work	Position	This paper	Relation
Goldbach 1742	the question posed	closed to one act	extends
Hardy and Littlewood 1923	the expected number of splits	not used	adjacent
Chen 1973	a prime plus a P_2	typed; does not give the binary	bounds
Helfgott 2013	three primes for every odd number	typed; does not give the binary	bounds
Oliveira e Silva, Herzog and Pardi 2014	verified to 4×10^{18}	carried, load-bearing, not reaching above	extends
Islam 2026c, the division	every row its proved part and remainder	the division at a height	extends
Islam 2026a, 2026b, 2026e	rows closed from existence alone	the drill carried to this row	extends

Table 3 | Positioning. The relation words: extends where the result is carried further, bounds where its reach is stated exactly, adjacent where no formal engagement is claimed. No prior position is contradicted.

19. CONCLUSION

Existence was given before the question was asked, and the paper proves exactly what that buys: the form on every frame and no value on any. It proves that existence read on the row is the value and, on the arithmetic frame, the Goldbach statement; that the statement follows from that reading by one act; that nothing escapes the act; that one even number without a split is the only refuter, and a finite one; that the kernel's own computation decides every even number to 2000; and that the certified region, the ternary theorem and Chen's theorem are sealed at their grades and do not reach the remainder. The verdict, in the words of Section 1, unchanged:

The Goldbach question is closed to one act of existence. Existence as given carries the form of the closure on every frame and holds in the realized world and the counter world alike. Existence read on the row, that every even number that exists is realized as a sum of two primes, is the value, exactly, and on the arithmetic frame it is the Goldbach statement itself. It is supplied by one act, at premise grade, as one field of one type, and the compiler prints the statement from it. Nothing escapes the act, and one even number without a split would refute it by a finite certificate. To 2000 the value is a theorem of the kernel's own computation, and to the certified height it is carried; above that height it stands on the act.

APPENDIX A · RECEIPTS

The kernel, GB_Existence_Closure.lean, compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0 (de Moura and Ullrich 2021), the executed region taking under a minute. It carries fifty-eight theorems: fifty-three print *does not depend on any axioms* and five print *depends on axioms*: [propext], each pinned by #guard_msgs so that a drifted cone fails the compile. Its SHA-256 is

82794234f9488bd89dd840259d897c6b
53777b1b9d1f7b5079a325a47075cbdb

and the Markdown master of this paper carries the kernel with a one-line extraction command and its manifest.

APPENDIX B · THE KERNEL

```

/-
GB_Existence_Closure.lean · the kernel of the Goldbach closure from existence alone

The Goldbach row closed on existence and the freedom arrow alone. Every theorem of
this file is
on no axiom at all, no propext, no Quot.sound, no Classical.choice, except five: the
bridge from
the remainder test to divisibility (small_divisor, isPrime_sound) and the three
statements that
pass through it (goldbach_gives_std, goldbach_iff_std, act_is_goldbach_std) rest on
propext
alone, through core Lean's own Nat.mod and Nat.div lemmas.
I The arithmetic: primality defined; the trial-division test, proved sound and
complete;
the search for a split, proved sound and complete; the Goldbach statement in
search form
and standard form, the two proved equivalent.
II The mirror  $p \mapsto n - p$ : an involution whose fixed point is  $n/2$ , the seat; splits
come in
mirror pairs.
III The frame of even numbers, the value, the two coherent worlds; the arithmetic
frame,
whose value is exactly the Goldbach statement.
IV Existence as given: the root, the arrow, the freedom bit; they force no value.
V Existence read on the row, the act: every even number that exists is realized as
a sum
of two primes. The act is the value; one act closes it; nothing escapes; one even
number without a split refutes it, a finite certificate.
VI The proved part: every even number from 4 to 2000 decided by the kernel's own
computation; the certified region carried as a field, load-bearing, not reaching
above
its height; the ternary and Chen realizations, carried abstractly, neither
giving the
binary.
VII The division at a height: the value is exactly its two halves.
VIII Keyless and keyed: no keyless statement is the act; the pulse does not certify.
IX Freedom: the record wall, the two-point fibre, the prime's shape, multiplicative
and
additive.
X The triaxial lock.
XI The record and the seed: no finite record forces the value; a uniform step
forces all.
XII The closure, whole.
-/

namespace GBClose

/-! ### subtraction, proved here on no axiom -/

theorem sub_cancel_right : ∀ n m : Nat, n + m - m = n
| _, 0 => rfl
| n, m + 1 => (Nat.succ_sub_succ (n + m) m).trans (sub_cancel_right n m)

theorem sub_cancel_left (n m : Nat) : n + m - n = m := by
rw [Nat.add_comm]; exact sub_cancel_right m n

theorem sub_add_back : ∀ n m : Nat, m ≤ n → n - m + m = n
| _, 0, _ => rfl
| 0, m + 1, h => absurd h (Nat.not_succ_le_zero m)
| n + 1, m + 1, h => by
rw [Nat.succ_sub_succ, Nat.add_succ]
exact congrArg Nat.succ (sub_add_back n m (Nat.le_of_succ_le_succ h))

/-! ## I · the arithmetic -/

def primeAux (n : Nat) : Nat → Nat → Bool
| 0, _ => true
| fuel + 1, d => if d * d > n then true else if n % d = 0 then false else primeAux n
fuel (d + 1)

```

```

/-- The trial-division test. -/
def isPrime (n : Nat) : Bool := if n < 2 then false else primeAux n n 2

/-- Primality, defined: at least two, and no remainder zero below it. -/
def Prime (p : Nat) : Prop := 2 ≤ p ∧ ∀ e, 2 ≤ e → e < p → p % e ≠ 0

theorem primeAux_spec (n : Nat) : ∀ fuel d, primeAux n fuel d = true →
  ∀ e, d ≤ e → e < d + fuel → e * e ≤ n → n % e ≠ 0
| 0, d, _, e, h1, h2, _ => absurd h2 (Nat.not_lt_of_le (by rw [Nat.add_zero]; exact h1))
| fuel + 1, d, h, e, h1, h2, h3 => by
  unfold primeAux at h
  by_cases g : d * d > n
  · -- every e ≥ d has e*e ≥ d*d > n
    exact absurd (Nat.lt_of_lt_of_le g (Nat.mul_le_mul h1 h1)) (Nat.not_lt_of_le h3)
  · rw [if_neg g] at h
  by_cases m : n % d = 0
  · rw [if_pos m] at h; exact Bool.noConfusion h
  · rw [if_neg m] at h
  cases Nat.eq_or_lt_of_le h1 with
  | inl he => exact he ▸ m
  | inr hl =>
    exact primeAux_spec n fuel (d + 1) h e hl
    (by rw [Nat.add_assoc, Nat.add_comm 1 fuel]; exact h2) h3

theorem small_divisor (n e : Nat) (h2 : 2 ≤ e) (hlt : e < n) (hd : n % e = 0) :
  ∃ f, 2 ≤ f ∧ f * f ≤ n ∧ n % f = 0 := by
  have dv : e | n := Nat.dvd_of_mod_eq_zero hd
  have hn : e * (n / e) = n := Nat.mul_div_cancel' dv
  have epos : 0 < e := Nat.lt_of_lt_of_le (by decide) h2
  have g2 : 2 ≤ n / e := by
    cases Nat.lt_or_ge (n / e) 2 with
    | inr h => exact h
    | inl h =>
      have : n / e ≤ 1 := Nat.le_of_lt_succ h
      have : n ≤ e := by
        have := Nat.mul_le_mul_left e this
        rw [hn, Nat.mul_one] at this; exact this
      exact absurd hlt (Nat.not_lt_of_le this)
  cases Nat.le_total e (n / e) with
  | inl hle => exact (e, h2, by have := Nat.mul_le_mul_left e hle; rw [hn] at this; exact this, hd)
  | inr hge =>
    refine (n / e, g2, ?, ?)
    · have := Nat.mul_le_mul_right (n / e) hge
      rw [hn] at this; exact this
    · exact Nat.mod_eq_zero_of_dvd (e, by rw [Nat.mul_comm (n / e) e]; exact hn.symm)

theorem isPrime_sound (p : Nat) (h : isPrime p = true) : Prime p := by
  unfold isPrime at h
  by_cases g : p < 2
  · rw [if_pos g] at h; exact Bool.noConfusion h
  · rw [if_neg g] at h
  refine (Nat.le_of_not_gt g, fun e h2 hlt hd => ?)
  obtain (f, f2, ff, fd) := small_divisor p e h2 hlt hd
  have fle : f < 2 + p := by
    have : f ≤ f * f := by
      have t := Nat.mul_le_mul_left f (Nat.le_trans (by decide : 1 ≤ 2) f2)
      rw [Nat.mul_one] at t; exact t
    exact Nat.lt_of_le_of_lt (Nat.le_trans this ff) (by rw [Nat.add_comm]; exact Nat.lt_add_of_pos_right (by decide))
  exact primeAux_spec p p 2 h f f2 fle ff fd

/-- The search: the least p with p and n - p prime and p ≤ n - p, below a bound. -/
def findAux (n : Nat) : Nat → Nat → Option Nat
| 0, _ => none
| fuel + 1, p =>
  if p * 2 > n then none
  else if isPrime p && isPrime (n - p) then some p
  else findAux n fuel (p + 1)

def findSplit (n : Nat) : Option Nat := findAux n n 2

/-- The search is sound: a found p is prime, its mirror is prime, and p is at most its mirror. -/
theorem findAux_sound (n : Nat) : ∀ fuel p q, findAux n fuel p = some q →
  isPrime q = true ∧ isPrime (n - q) = true ∧ q * 2 ≤ n
| 0, _, _, h => Option.noConfusion h
| fuel + 1, p, q, h => by
  unfold findAux at h
  by_cases h1 : p * 2 > n
  · rw [if_pos h1] at h; exact Option.noConfusion h
  · rw [if_neg h1] at h
  cases h2 : isPrime p with
  | false =>
    rw [h2] at h
    change (if false = true then some p else findAux n fuel (p + 1)) = some q at h
    rw [if_neg Bool.false_ne_true] at h
    exact findAux_sound n fuel (p + 1) q h
  | true =>
    cases h3 : isPrime (n - p) with
    | false =>
      rw [h2, h3] at h
      change (if false = true then some p else findAux n fuel (p + 1)) = some q at h
      rw [if_neg Bool.false_ne_true] at h
      exact findAux_sound n fuel (p + 1) q h
    | true =>
      rw [h2, h3] at h
      change (if true = true then some p else findAux n fuel (p + 1)) = some q at h
      rw [if_pos rfl] at h

have hq : p = q := Option.some.inj h
exact (hq ▸ h2, hq ▸ h3, hq ▸ Nat.le_of_not_gt h1)

theorem findSplit_sound (n q : Nat) (h : findSplit n = some q) :
  isPrime q = true ∧ isPrime (n - q) = true ∧ q * 2 ≤ n :=
  findAux_sound n n 2 q h

theorem primeAux_false (n : Nat) : ∀ fuel d, primeAux n fuel d = false →
  ∃ e, d ≤ e ∧ e * e ≤ n ∧ n % e = 0
| 0, _, h => Bool.noConfusion h
| fuel + 1, d, h => by
  unfold primeAux at h
  by_cases g : d * d > n
  · rw [if_pos g] at h; exact Bool.noConfusion h
  · rw [if_neg g] at h
  by_cases m : n % d = 0
  · exact (d, Nat.le_refl d, Nat.le_of_not_gt g, m)
  · rw [if_neg m] at h
  match primeAux_false n fuel (d + 1) h with
  | (e, he, h2, h3) => exact (e, Nat.le_of_succ_le he, h2, h3)

theorem isPrime_complete (p : Nat) (h : Prime p) : isPrime p = true := by
  unfold isPrime
  rw [if_neg (Nat.not_lt_of_le h.1)]
  cases hb : primeAux p p 2 with
  | true => rfl
  | false =>
    match primeAux_false p p 2 hb with
    | (e, he, hee, hm) =>
      have elt : e < p := by
        have : e * 2 ≤ e * e := Nat.mul_le_mul_left e he
        have : e < e * 2 := by
          rw [Nat.mul_two]; exact Nat.lt_add_of_pos_right (Nat.lt_of_lt_of_le (by decide) he)
        exact Nat.lt_of_lt_of_le this (Nat.le_trans (Nat.mul_le_mul_left e he) hee)
      exact absurd hm (h.2 e he elt)

/-- The search is complete: if it returns nothing, no split lies in its range. -/
theorem findAux_complete (n : Nat) : ∀ fuel p, findAux n fuel p = none → n < 2 * (p + fuel) →
  ∀ q, p ≤ q + q * 2 ≤ n → (isPrime q = true ∧ isPrime (n - q) = true)
| 0, p, _, hb, q, hq, hq2, _ => by
  rw [Nat.add_zero] at hb
  have : 2 * p ≤ q * 2 := by rw [Nat.mul_comm]; exact Nat.mul_le_mul_right 2 hq
  exact absurd (Nat.lt_of_lt_of_le hb (Nat.le_trans this hq2)) (Nat.lt_irrefl n)
| fuel + 1, p, h, hb, q, hq, hq2, hpq => by
  unfold findAux at h
  by_cases g : p * 2 > n
  · have : p * 2 ≤ q * 2 := Nat.mul_le_mul_right 2 hq
    exact absurd (Nat.lt_of_lt_of_le g (Nat.le_trans this hq2)) (Nat.lt_irrefl n)
  · rw [if_neg g] at h
  cases Nat.eq_or_lt_of_le hq with
  | inl he =>
    subst he
    rw [hpq.1, hpq.2] at h
    change (if true = true then some p else findAux n fuel (p + 1)) = none at h
    rw [if_pos rfl] at h; exact Option.noConfusion h
  | inr hl =>
    cases h2 : isPrime p && isPrime (n - p) with
    | true =>
      rw [h2] at h; change (if true = true then some p else findAux n fuel (p + 1)) = none at h
      rw [if_pos rfl] at h; exact Option.noConfusion h
    | false =>
      rw [h2] at h; change (if false = true then some p else findAux n fuel (p + 1)) = none at h
      rw [if_neg Bool.false_ne_true] at h
      exact findAux_complete n fuel (p + 1) h (by rw [Nat.add_assoc, Nat.add_comm 1 fuel]; exact hb)
      q hl hq2 hpq

/-- An even number at least four. -/
def EvenAtLeast4 (n : Nat) : Prop := 4 ≤ n ∧ n % 2 = 0

/-- THE GOLDBACH STATEMENT, in the kernel's arithmetic. -/
def Goldbach : Prop := ∀ n, EvenAtLeast4 n → ∃ p q, findSplit n = some q

/-- The Goldbach statement in the standard form: two primes summing to n. -/
def GoldbachStd : Prop := ∀ n, EvenAtLeast4 n → ∃ p q, Prime p ∧ Prime q ∧ p + q = n

/-- The search form gives the standard form. -/
theorem goldbach_gives_std (h : Goldbach) : GoldbachStd := fun n hn =>
  match h n hn with
  | (q, hq) =>
    have s := findSplit_sound n q hq
    have hle : q ≤ n := Nat.le_trans (Nat.le_mul_of_pos_right q (by decide)) s.2.2
    (q, n - q, isPrime_sound q s.1, isPrime_sound (n - q) s.2.1, by rw [Nat.add_comm]; exact sub_add_back n q hle)

/-- The standard form gives the search form: the search is complete. -/
theorem std_gives_goldbach (h : GoldbachStd) : Goldbach := fun n hn =>
  match h n hn with
  | (p, q, hp, hq, hs) =>
    match hf : findSplit n with
    | some r => (r, rfl)
    | none => by
      cases Nat.le_total p q with
      | inl hle =>
        have h2 : p * 2 ≤ n := by rw [Nat.mul_two, ← hs]; exact Nat.add_le_add_left hle p
        have hsub : n - p = q := by rw [← hs]; exact sub_cancel_left p q
        exact absurd (isPrime_complete p hp, hsub ▸ isPrime_complete q hq)

```

```

      (findAux_complete n n 2 hf (by
        show n < 2 * (2 + n)
        rw [Nat.add_comm, Nat.mul_add]
        exact Nat.lt_of_le_of_lt (Nat.le_mul_of_pos_left n (by decide))
        (Nat.lt_add_of_pos_right (by decide)))
      p hp.1 h2)
| inr hle =>
  have h2 : q * 2 ≤ n := by rw [Nat.mul_two, ← hs, Nat.add_comm p q]; exact
  Nat.add_le_add_left hle q
  have hsub : n - q = p := by rw [← hs]; exact sub_cancel_right p q
  exact absurd (isPrime_complete q hq, hsub ▸ isPrime_complete p hp)
  (findAux_complete n n 2 hf (by
    show n < 2 * (2 + n)
    rw [Nat.add_comm, Nat.mul_add]
    exact Nat.lt_of_le_of_lt (Nat.le_mul_of_pos_left n (by decide))
    (Nat.lt_add_of_pos_right (by decide)))
    q hq.1 h2)

/- THE TWO FORMS ARE EQUIVALENT. -/
theorem goldbach_iff_std : Goldbach ↔ GoldbachStd := (goldbach_gives_std,
std_gives_goldbach)

/-! ## II · the mirror and its seat -/

def mirror (n p : Nat) : Nat := n - p

theorem mirror_involution (n p : Nat) (h : p ≤ n) : mirror n (mirror n p) = p := by
  show n - (n - p) = p
  have e : n - p + p = n := sub_add_back n p h
  have t : n - p + p - (n - p) = p := sub_cancel_left (n - p) p
  rw [e] at t
  exact t

/- A split mirrors to a split: if p and n - p are prime, so are n - p and n - (n - p). -/
theorem split_mirrors (n p : Nat) (h : p ≤ n) (hp : isPrime p = true) (hq : isPrime (n - p) = true) :
  isPrime (mirror n p) = true ∧ isPrime (n - mirror n p) = true :=
  (hq, by show isPrime (n - (n - p)) = true; rw [show n - (n - p) = p from
  mirror_involution n p h]; exact hp)

/- The seat: the mirror fixes p exactly when p + p = n. -/
theorem seat_of_mirror (n p : Nat) (h : p ≤ n) : mirror n p = p ↔ p + p = n :=
  (fun e => by
    have : n - p + p = n := sub_add_back n p h
    rw [show mirror n p = n - p from rfl] at e; rw [e] at this; exact this,
    fun e => by show n - p = p; rw [← e]; exact sub_cancel_right p p)

/-! ## III · the frame, and the arithmetic frame -/

structure Frame where
  D : Type
  realize : D → Option Nat

def Value (F : Frame) : Prop := ∀ d, ∃ q, F.realize d = some q

def calm : Frame := (Unit, fun _ => some 3)
def counter : Frame := (Unit, fun _ => none)

theorem calm_value : Value calm := fun _ => (3, rfl)
theorem counter_fails : ¬ Value counter := fun h => match h () with
| (_, hn) => Option.noConfusion hn

/- The arithmetic frame: the even numbers at least four, each realized by the search. -/
def arith : Frame := ({ n : Nat // EvenAtLeast4 n }, fun d => findSplit d.1)

/- THE ARITHMETIC FRAME'S VALUE IS EXACTLY THE GOLDBACH STATEMENT. -/
theorem arith_value_iff : Value arith ↔ Goldbach :=
  (fun h n hn => h (n, hn), fun h d => h d.1 d.2)

/-! ## IV · existence as given -/

def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

theorem root_given : Root Empty (fun e => nomatch e) := fun e => nomatch e

theorem root_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U → Int), Root U ΔE → S) :
S :=
  h Empty (fun e => nomatch e) root_given

def Arrow : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a

theorem arrow_given : Arrow := fun _ => (id, fun _ => rfl)

theorem freedom_given : ∀ w : Bool, (!w) ≠ w
| true => Bool.noConfusion
| false => Bool.noConfusion

theorem given_is_not_the_value (P : Prop) : ¬ ∀ F : Frame, (P ↔ Value F) :=
  fun h => counter_fails ((h counter).mp ((h calm).mpr calm_value))

theorem arrow_forces_nothing : ¬ ∀ F : Frame, Arrow → Value F :=
  fun h => counter_fails (h counter arrow_given)

theorem given_in_both_worlds : (Arrow ∧ Value calm) ∧ (Arrow ∧ ¬ Value counter) :=
  ((arrow_given, calm_value), (arrow_given, counter_fails))

/-! ## V · existence read on the row: the act -/

```

```

/- Existence read on the row: every even number that exists is realized as a sum of two
primes. -/
def Realized (F : Frame) : Prop := ∀ d, ∃ q, F.realize d = some q

theorem act_is_the_value (F : Frame) : Realized F ↔ Value F :=
  (fun h d => h d, fun h d => h d)

theorem act_is_keyed : Realized calm ∧ ¬ Realized counter := (calm_value, counter_fails)

/- On the arithmetic frame the act is exactly the Goldbach statement. -/
theorem act_is_goldbach : Realized arith ↔ Goldbach :=
  (act_is_the_value arith).trans arith_value_iff

/- On the arithmetic frame the act is exactly the Goldbach statement in its standard
form. -/
theorem act_is_goldbach_std : Realized arith ↔ GoldbachStd := act_is_goldbach.trans
goldbach_iff_std

structure ActualEvens where
  F : Frame
  supply : Realized F

theorem goldbach_from_existence (A : ActualEvens) : Value A.F :=
  (act_is_the_value A.F).mp A.supply

/- THE GOLDBACH STATEMENT FROM EXISTENCE, BY ONE ACT, on the arithmetic frame. -/
theorem goldbach_from_the_act (h : Realized arith) : Goldbach := act_is_goldbach.mp h

theorem supply_iff (F : Frame) : Nonempty { A : ActualEvens // A.F = F } ↔ Value F :=
  (fun ((A, hA)) => hA ▸ goldbach_from_existence A,
  fun h => (((F, (act_is_the_value F).mpr h), rfl)))

theorem every_even_lands (F : Frame) (d : F.D) :
  (∃ q, F.realize d = some q) ∨ F.realize d = none :=
  match F.realize d with
  | some q => 0r.inl (q, rfl)
  | none => 0r.inr rfl

theorem gates_exclusive (F : Frame) (d : F.D) :
  ¬ ((∃ q, F.realize d = some q) ∧ F.realize d = none) :=
  fun (⟦_, h⟧, h') => Option.noConfusion (h'.symm.trans h)

theorem nothing_escapes (A : ActualEvens) : ∀ d, ∃ q, A.F.realize d = some q := A.supply

/- One even number with no split is a finite certificate refuting the Goldbach
statement. -/
theorem counterexample_refutes (n : Nat) (hn : EvenAtLeast4 n) (h : findSplit n = none) :
  ¬ Goldbach :=
  fun g => match g n hn with
  | (_, hq) => Option.noConfusion (h.symm.trans hq)

/-! ## VI · the proved part -/

/- The executed check: every even number 2m with 2 ≤ m ≤ k has a split. -/
def checkAux : Nat → Bool
| 0 => true
| m + 1 => (decide (m + 1 < 2) || (findSplit (2 * (m + 1))).isSome) && checkAux m

def checkUpTo (N : Nat) : Bool := checkAux (N / 2)

/- What the executed check means: every even number 2m with 2 ≤ m ≤ k has a split. -/
theorem check_means : ∀ k m, checkAux k = true → m ≤ k → 2 ≤ m → (findSplit (2 *
m)).isSome = true
| 0, m, _, hm, h2 => absurd (Nat.le_trans h2 hm) (by decide)
| k + 1, m, h, hm, h2 => by
  unfold checkAux at h
  cases ha : (decide (k + 1 < 2) || (findSplit (2 * (k + 1))).isSome) with
  | false => rw [ha, Bool.false_and] at h; exact Bool.noConfusion h
  | true =>
    cases hb : checkAux k with
    | false => rw [ha, hb] at h; exact Bool.noConfusion h
    | true =>
      cases Nat.eq_or_lt_of_le hm with
      | inr hl => exact check_means k m hb (Nat.le_of_lt_succ hl) h2
      | inl he =>
        subst he
        cases hd : decide (k + 1 < 2) with
        | true => exact absurd h2 (Nat.not_le_of_gt (of_decide_eq_true hd))
        | false => rw [hd, Bool.false_or] at ha; exact ha

set_option maxRecDepth 100000 in
/- THE EXECUTED REGION: every even number from 4 to 2000 is a sum of two primes,
decided by the
kernel's own computation. -/
theorem executed_to_2000 : checkUpTo 2000 = true := by decide

/- The certified region, carried as a field: every even number from 4 to the height has
a split
(the cited verification to 4 × 1018). -/
structure Certified where
  H : Nat
  cert : ∀ n, EvenAtLeast4 n → n ≤ H → ∃ q, findSplit n = some q

theorem below_height_decided (C : Certified) (n : Nat) (hn : EvenAtLeast4 n) (hH : n ≤
C.H) :
  ∃ q, findSplit n = some q := C.cert n hn hH

/- An abstract certified frame: a height, and a realization certified below it. -/
structure CertFrame where
  F : Frame
  ht : F.D → Nat

```

```

H      : Nat
cert   : ∀ d, ht d ≤ H → ∃ q, F.realize d = some q

def certCounter : CertFrame := ((Nat, fun d => if d ≤ 10 then some 3 else none), fun d
=> d, 10,
  fun d h => (3, by show (if d ≤ 10 then some 3 else none) = some 3; rw [if_pos h]))

/- THE CERTIFICATE DOES NOT REACH ABOVE ITS HEIGHT. -/
theorem certificate_does_not_reach_above : ¬ Value certCounter.F :=
  fun h => match h (show certCounter.F.D from (11 : Nat)) with
  | (_, hq) => by
    have e : certCounter.F.realize (show certCounter.F.D from (11 : Nat)) = none := by
      show (if (11 : Nat) ≤ 10 then some 3 else none) = none
    rw [if_neg (by decide : ¬ (11 ≤ 10))]
    exact Option.noConfusion (e.symm.trans hq)

structure CertFrameNoCert where
  F : Frame
  ht : F.D → Nat
  H : Nat

def bareCert : CertFrameNoCert := (counter, fun _ => 4, 10)

/- THE CERTIFICATE IS LOAD-BEARING: without it an even number below the height goes
unrealized. -/
theorem certificate_is_load_bearing : bareCert.ht () ≤ bareCert.H ∧ ∃ q,
bareCert.F.realize () = some q :=
  (by decide, fun (_, h) => Option.noConfusion h)

/- A frame with two abstract realizations beside the binary one, standing for the cited
ternary
theorem (three primes for every odd number) and Chen's theorem (a prime plus a prime or
semiprime
for every large even number); neither statement is formalized here, only the shape they
share. -/
structure Weaker where
  F      : Frame
  ternary : F.D → Option Nat
  chen   : F.D → Option Nat
  tern    : ∀ d, ∃ t, ternary d = some t
  chenAll : ∀ d, ∃ c, chen d = some c

def weakerCounter : Weaker := (counter, fun _ => some 3, fun _ => some 3,
  fun _ => (3, rfl), fun _ => (3, rfl))

/- NEITHER THE TERNARY THEOREM NOR CHEN'S THEOREM GIVES THE BINARY VALUE. -/
theorem weaker_do_not_give_binary :
  (∀ d, ∃ t, weakerCounter.ternary d = some t) ∧ (∀ d, ∃ c, weakerCounter.chen d =
  some c) ∧
  ¬ Value weakerCounter.F :=
  (weakerCounter.tern, weakerCounter.chenAll, counter_fails)

/-! ## VII · the division at a height -/

def ValueOn (F : Frame) (cut : F.D → Bool) (b : Bool) : Prop :=
  ∀ d, cut d = b → ∃ q, F.realize d = some q

theorem row_split (F : Frame) (cut : F.D → Bool) :
  Value F ⇔ ValueOn F cut true ∧ ValueOn F cut false :=
  (fun h => (fun d _ => h d, fun d _ => h d),
  fun (ht, hf) d => match hc : cut d with
  | true => ht d hc
  | false => hf d hc)

/- The certified half holds and the remainder fails in the certified counter world. -/
theorem remainder_not_forced :
  ValueOn certCounter.F (fun d => Nat.ble d 10) true ∧
  ¬ ValueOn certCounter.F (fun d => Nat.ble d 10) false :=
  (fun d h => certCounter.cert d (Nat.le_of_ble_eq_true h),
  fun h => certificate_does_not_reach_above (fun d =>
  match hc : Nat.ble d 10 with
  | true => certCounter.cert d (Nat.le_of_ble_eq_true hc)
  | false => h d hc))

/-! ## VIII · keyless and keyed -/

def Keyed (P : Frame → Prop) : Prop := ∃ F, ¬ P F

theorem value_is_keyed : Keyed Value := (counter, counter_fails)

theorem no_keyless_statement_is_the_act (P : Prop) : ¬ ∀ F : Frame, (P ⇔ Realized F) :=
  fun h => given_is_not_the_value P (fun F => (h F).trans (act_is_the_value F))

theorem pulse_does_not_certify : Root Empty (fun e => nomatch e) ∧ ¬ Value counter :=
  (root_given, counter_fails)

/-! ## IX · freedom -/

def kineticRecord (w : Bool) : Nat := 0

theorem record_wall : ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w :=
  fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

theorem fibre_is_two : ({false, true}.filter (fun w => kineticRecord w == kineticRecord
true)).length = 2 := by
  decide

def mulFibre (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).flatMap fun a =>
  (List.range (n + 1)).filterMap fun b => if a * b = n then some (a, b) else none

theorem prime_shape :
  mulFibre 7 = [(1, 7), (7, 1)] ∧ (mulFibre 7).all (fun p => p.1 != p.2) = true := by
  decide

/- The additive fibre: the ordered splits of n into two primes. -/
def addFibre (n : Nat) : List (Nat × Nat) :=
  (List.range (n + 1)).filterMap fun p => if isPrime p && isPrime (n - p) then some (p,
  n - p) else none

/- The additive shape: the splits of 10 are one mirror pair off the seat and one point
on it,
and the splits of 14 are one mirror pair off the seat and one on it. -/
theorem additive_shape :
  addFibre 10 = [(3, 7), (5, 5), (7, 3)] ∧ addFibre 14 = [(3, 11), (7, 7), (11, 3)] ∧
  addFibre 16 = [(3, 13), (5, 11), (11, 5), (13, 3)] := by
  decide

/-! ## X · the triaxial lock -/

def dot2 (r x : Bool × Bool × Bool) : Bool :=
  xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool × Bool × Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b => [false, true].map fun c
=> (a, b, c)

def nonzero : List (Bool × Bool × Bool) := cube.filter (fun r => r != (false, false,
false))

def solCount (rows : List ((Bool × Bool × Bool) × Bool)) : Nat :=
  (cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x == rt.2))).length

def xor3 (a b : Bool × Bool × Bool) : Bool × Bool × Bool :=
  (xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
  nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
  [false, true].all (fun t1 => [false, true].all (fun t2 =>
  solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
  nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun r3 =>
  r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
  [false, true].all (fun t1 => [false, true].all (fun t2 => [false, true].all (fun
  t3 =>
  solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true := by decide

/-! ## XI · the record and the seed -/

def staged (n : Nat) : Frame := (Nat, fun d => if d < n then some 3 else none)

theorem finite_record_never_forces (n : Nat) :
  (∀ d, d < n → ∃ q, (staged n).realize d = some q) ∧ ¬ Value (staged n) :=
  (fun d hd => (3, by show (if d < n then some 3 else none) = some 3; rw [if_pos hd]),
  fun h => match h n with
  | (_, hk) => by
    have e : (staged n).realize n = none := by
      show (if n < n then some 3 else (none : Option Nat)) = none
    rw [if_neg (Nat.lt_irrefl n)]
    exact Option.noConfusion (e.symm.trans hk))

structure Ladder where
  Decided : Nat → Prop
  base    : Decided 0
  mono    : ∀ r s, Decided s → r ≤ s → Decided r
  δ       : Nat → Nat
  step    : ∀ r, Decided r → Decided (r + δ r)

theorem uniform_step_forces_all (L : Ladder) (hδ : ∀ r, 1 ≤ L.δ r) : ∀ r, L.Decided r
| 0 => L.base
| r + 1 => L.mono (r + 1) (r + L.δ r) (L.step r (uniform_step_forces_all L hδ r))
  (Nat.add_le_add_left (hδ r) r)

/-! ## XII · the closure, whole -/

theorem goldbach_closure :
  (∀ P : Prop, ¬ ∀ F : Frame, (P ⇔ Value F)) ∧
  (Realized arith ⇔ Goldbach) ∧
  (∀ n, EvenAtLeast4 n → findSplit n = none → ¬ Goldbach) ∧
  checkUpTo 2000 = true ∧
  (∀ (C : Certified) (n : Nat), EvenAtLeast4 n → n ≤ C.H → ∃ q, findSplit n = some q) ∧
  ¬ Value certCounter.F ∧
  ¬ Value weakerCounter.F ∧
  Keyed Value :=
  (given_is_not_the_value, act_is_goldbach, counterexample_refutes, executed_to_2000,
  below_height_decided, certificate_does_not_reach_above,
  weaker_do_not_give_binary.2.2,
  value_is_keyed)

end GBClose

/-! ## Cones, pinned as printed: fifty-three theorems on no axiom, five on propext alone
-/
/- info: 'GBClose.sub_cancel_right' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.sub_cancel_right
/- info: 'GBClose.sub_cancel_left' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.sub_cancel_left
/- info: 'GBClose.sub_add_back' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.sub_add_back
/- info: 'GBClose.primeAux_spec' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.primeAux_spec

```

```

/-- info: 'GBClose.small_divisor' depends on axioms: [propxet] -/
#guard_msgs in #print axioms GBClose.small_divisor
/-- info: 'GBClose.isPrime_sound' depends on axioms: [propxet] -/
#guard_msgs in #print axioms GBClose.isPrime_sound
/-- info: 'GBClose.findAux_sound' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.findAux_sound
/-- info: 'GBClose.findSplit_sound' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.findSplit_sound
/-- info: 'GBClose.primeAux_false' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.primeAux_false
/-- info: 'GBClose.isPrime_complete' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.isPrime_complete
/-- info: 'GBClose.findAux_complete' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.findAux_complete
/-- info: 'GBClose.goldbach_gives_std' depends on axioms: [propxet] -/
#guard_msgs in #print axioms GBClose.goldbach_gives_std
/-- info: 'GBClose.std_gives_goldbach' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.std_gives_goldbach
/-- info: 'GBClose.goldbach_iff_std' depends on axioms: [propxet] -/
#guard_msgs in #print axioms GBClose.goldbach_iff_std
/-- info: 'GBClose.mirror_involution' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.mirror_involution
/-- info: 'GBClose.split_mirrors' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.split_mirrors
/-- info: 'GBClose.seat_of_mirror' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.seat_of_mirror
/-- info: 'GBClose.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.calm_value
/-- info: 'GBClose.counter_fails' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.counter_fails
/-- info: 'GBClose.arith_value_iff' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.arith_value_iff
/-- info: 'GBClose.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.root_given
/-- info: 'GBClose.root_conservative' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.root_conservative
/-- info: 'GBClose.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.arrow_given
/-- info: 'GBClose.freedom_given' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.freedom_given
/-- info: 'GBClose.given_is_not_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.given_is_not_the_value
/-- info: 'GBClose.arrow_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.arrow_forces_nothing
/-- info: 'GBClose.given_in_both_worlds' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.given_in_both_worlds
/-- info: 'GBClose.act_is_the_value' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.act_is_the_value
/-- info: 'GBClose.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.act_is_keyed
/-- info: 'GBClose.act_is_goldbach' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.act_is_goldbach
/-- info: 'GBClose.act_is_goldbach_std' depends on axioms: [propxet] -/
#guard_msgs in #print axioms GBClose.act_is_goldbach_std
/-- info: 'GBClose.goldbach_from_existence' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.goldbach_from_existence
/-- info: 'GBClose.goldbach_from_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.goldbach_from_the_act
/-- info: 'GBClose.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.supply_iff
/-- info: 'GBClose.every_even_lands' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.every_even_lands
/-- info: 'GBClose.gates_exclusive' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.gates_exclusive
/-- info: 'GBClose.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.nothing_escapes
/-- info: 'GBClose.counterexample_refutes' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.counterexample_refutes
/-- info: 'GBClose.check_means' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.check_means
/-- info: 'GBClose.executed_to_2000' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.executed_to_2000
/-- info: 'GBClose.below_height_decided' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.below_height_decided
/-- info: 'GBClose.certificate_does_not_reach_above' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.certificate_does_not_reach_above
/-- info: 'GBClose.certificate_is_load_bearing' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.certificate_is_load_bearing
/-- info: 'GBClose.weaker_do_not_give_binary' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.weaker_do_not_give_binary
/-- info: 'GBClose.row_split' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.row_split
/-- info: 'GBClose.reminder_not_forced' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.reminder_not_forced
/-- info: 'GBClose.value_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.value_is_keyed
/-- info: 'GBClose.no_keyless_statement_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.no_keyless_statement_is_the_act
/-- info: 'GBClose.pulse_does_not_certify' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.pulse_does_not_certify
/-- info: 'GBClose.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.record_wall
/-- info: 'GBClose.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.fibre_is_two
/-- info: 'GBClose.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.prime_shape
/-- info: 'GBClose.additive_shape' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.additive_shape
/-- info: 'GBClose.two_axes_leave_two' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.two_axes_leave_two
/-- info: 'GBClose.three_axes_lock_one' does not depend on any axioms -/

```

```

#guard_msgs in #print axioms GBClose.three_axes_lock_one
/-- info: 'GBClose.finite_record_never_forces' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.finite_record_never_forces
/-- info: 'GBClose.uniform_step_forces_all' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.uniform_step_forces_all
/-- info: 'GBClose.goldbach_closure' does not depend on any axioms -/
#guard_msgs in #print axioms GBClose.goldbach_closure

```

AUTHOR’S PROVENANCE AND METHOD DIS-CLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the reference below.

Author and method. Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author’s positions on instruction; it holds no authorship. One claim was held at the model’s insistence: the five theorems that pass through the arithmetic bridge are graded on propext and named, rather than the kernel being called axiom-free throughout.

The register. In the programme’s vocabulary: the form, the act’s identity with the value and the statement, the universal closure, the executed region and the typed citations stand at $[\{\epsilon\}, T]$, fifty-three theorems on no axiom and five on propext alone; the triaxial reading at $[\{\epsilon\}, S]$; the value above the certified height at premise grade on the act; $\Delta M = 0$ on the cited number theory. The act is the row’s least-erasure posit read as existence realized; the row carries no defeater. The freedom cut, prime-as-freedom in its multiplicative and additive faces, and the triaxial lock are applied on the row, with the lessons of the Riemann, P versus NP, Navier–Stokes, Hodge and Yang–Mills closures combined, and the seat of the mirror read as the programme’s seat.

Audit. This edition has not yet been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author’s Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapients/Trisduction.

REFERENCES

- Chen, J. R. 1973. On the representation of a larger even integer as the sum of a prime and the product of at most two primes. *Scientia Sinica* 16: 157--176.
- de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE* 28, 625--635.
- Goldbach, C. 1742. Letter to L. Euler, 7 June 1742. In P.-H. Fuss (ed.), *Correspondance mathématique et physique de quelques célèbres géomètres du XVIIIème siècle*, vol. 1. St. Petersburg, 1843.
- Hardy, G. H. and J. E. Littlewood. 1923. Some problems of ‘Partitio Numerorum’; III: On the expression of a number as a sum of primes. *Acta Mathematica* 44: 1--70.

- Helfgott, H. A. 2013. The ternary Goldbach conjecture is true. arXiv:1312.7748.
- Islam, M. F. 2026a. A formal completed proof of the Navier--Stokes closure from existence alone. Zenodo. doi:10.5281/zenodo.22670344.
- Islam, M. F. 2026b. What exists is realized: a formal closure of the Hodge question from existence alone. Zenodo. doi:10.5281/zenodo.22701510.
- Islam, M. F. 2026c. The cut-agnostic division theorem: why every open problem is exactly its proved part and its unicorn. Zenodo. doi:10.5281/zenodo.22954862.
- Islam, M. F. 2026d. The veil and the wall. Zenodo. doi:10.5281/zenodo.21389758.
- Islam, M. F. 2026e. The Riemann Hypothesis closed to one named bit and proved from unconditional least erasure by one act, with an empty axiom cone. Zenodo. doi:10.5281/zenodo.23034066.
- Islam, M. F. 2026f. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.
- Islam, M. F. 2026g. PhysOS^T: the Trisduction physical operating system. Zenodo. doi:10.5281/zenodo.23117440.
- Islam, M. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTTG. Master reference, continuously updated at the same location.
- Oliveira e Silva, T., S. Herzog and S. Pardi. 2014. Empirical verification of the even Goldbach conjecture and computation of prime gaps up to $4 \cdot 10^{18}$. *Mathematics of Computation* 83: 2033--2060.

End of manuscript